Chapter 9 — Operator Fusion and the Compiler Stack

> *"The most important optimization in deep learning compilers is operator fusion. Every other optimization is either a special case of fusion or a precondition for it."* — Horace He, "Making Deep Learning Go Brrr From First Principles"

Chapter Overview

A naive PyTorch model executes each operation — every RMSNorm, every activation function, every residual add — as a separate GPU kernel. Each kernel launch carries CPU overhead. Each intermediate result makes a round-trip through HBM. At LLaMA-3 70B scale: 160 un-fused RMSNorm calls alone waste 21 GB of HBM traffic and 6 ms per step. The result is a model running at 10–20% of theoretical peak even with perfect algorithmic design.

The compiler stack closes this gap. Through operator fusion, graph optimization, and code generation, modern compilers — TorchInductor, XLA, TensorRT — routinely deliver 1.3–2.6× speedups over naive dispatch without changing the model at all.

Sections in this chapter:

- 9.1 The Three Performance Regimes — Diagnosing Before Compiling
- 9.2 Operator Fusion — Types, Mechanics, and HBM Savings
- 9.3 The PyTorch 2.x Compiler Stack — Dynamo → AOTAutograd → Inductor → Triton
- 9.4 XLA — The Functional Compiler (JAX / TPU)
- 9.5 TensorRT and TRT-LLM — Offline Production Compilation
- 9.6 NVFuser — NVIDIA's Runtime Fusion Engine
- 9.7 Graph-Level Optimizations in Depth
- 9.8 Min-Cut Recomputation — Fusion and Activation Checkpointing
- 9.9 CUDA Graphs — Eliminating CPU Dispatch Overhead
- 9.10 Compiler Selection Framework — When to Use What
- 9.11 Key Takeaways and Review Questions

9.1 The Three Performance Regimes — Diagnosing Before Compiling

Before applying any compiler optimization, you must classify the bottleneck.

Applying the wrong fix wastes engineering time and may make performance worse.

```
THE THREE REGIMES AND WHAT COMPILERS TARGET:

REGIME 1 — MEMORY-BANDWIDTH-BOUND:
  Symptom:  HBM bandwidth near saturation; TFLOPS utilization low.
  Root cause: Too many small ops reading/writing HBM for tiny amounts of compute.
  Compiler target: OPERATOR FUSION — merge ops to eliminate HBM round-trips.
  Example: RMSNorm (3 HBM ops) + ReLU (2 HBM ops) → 1 fused kernel (2 ops).
  Metric: dram__bytes_read.avg vs theoretical minimum.
  When NOT a compiler problem: it's compute-limited with correct bandwidth usage.

REGIME 2 — COMPUTE-BOUND:
  Symptom:  TFLOPS utilization > 70%; HBM below saturation.
  Root cause: GEMM is the bottleneck; TCs are the limiting resource.
  Compiler target: KERNEL SELECTION, TILE TUNING, PRECISION ASSIGNMENT.
  Example: TensorRT benchmarks 50+ cuBLAS variants and picks the fastest.
  Metric: sm__pipe_tensor_cycles_active.avg.pct_of_peak.
  When NOT a compiler problem: hardware ceiling reached; need faster hardware.

REGIME 3 — OVERHEAD-BOUND:
  Symptom:  GPU utilization fluctuates; gaps between kernels in nsys timeline.
  Root cause: CPU kernel launch overhead dominates GPU execution time.
  Compiler target: CUDA GRAPHS, GRAPH BREAK ELIMINATION.
  Example: 2,100 kernel launches × 5 μs = 10.5 ms CPU overhead per decode step.
  Metric: Nsys timeline gaps; GPU idle % between kernels.
  When NOT a compiler problem: data loading stall (fix the pipeline, not the
graph).

REGIME IDENTIFICATION DECISION TREE:
  Step 1: Run nsys. Is GPU idle > 10% between compute kernels?
    YES → overhead-bound → apply CUDA Graphs / torch.compile reduce-overhead
  Step 2: ncu roofline. Is the kernel dot below the bandwidth roof?
    YES → memory-bound → apply operator fusion
  Step 3: ncu roofline. Is the kernel dot near the compute roof?
    YES → compute-bound → apply kernel selection / FP8
  None of above: likely a new bottleneck exposed after fixing a previous one.
```

9.2 Operator Fusion — Types, Mechanics, and HBM Savings

9.2.1 The Three Types of Fusion

```
TYPE 1 — VERTICAL FUSION (chain of sequential ops):

  Before: x → [Op A] → tmp1 → HBM → [Op B] → tmp2 → HBM → [Op C] → out
  After:  x → [Op A + B + C fused] → out

  Requirements:
    - B consumes ALL of A's output (not shared)
    - B is ready to compute as soon as A produces each element
    - Total intermediate state fits in registers/SMEM

  Classic LLM examples:
    GEMM + add_bias + RMSNorm + GELU → one kernel
    Residual_add + RMSNorm           → one kernel
    SiLU_gate + elementwise_mult     → one kernel (SwiGLU gate computation)

  HBM traffic reduction:
    Unfused: each intermediate writes N elements, next reads N elements
    Fused:   intermediate stays in registers; only input and output touch HBM

TYPE 2 — HORIZONTAL FUSION (parallel ops merged into wider kernel):

  Before:
    Q_proj = linear(x, W_Q)    # separate kernel
    K_proj = linear(x, W_K)    # separate kernel
    V_proj = linear(x, W_V)    # separate kernel
    (Each reads x from HBM separately)

  After:
    QKV = linear(x, W_QKV)    # one wider GEMM
    Q, K, V = split(QKV, 3)   # cheap split (no data movement)
    (x is read from HBM exactly ONCE instead of three times)

  HBM traffic reduction:
    Before: 3 reads of x = 3 × (S × d_model × 2B) = 3 × 67 MB = 201 MB
    After:  1 read of x = 67 MB
    Savings: 134 MB per transformer layer (3× reduction on x reads)
    At LLaMA-3 70B, 80 layers: 80 × 134 MB = 10.7 GB per forward pass saved

TYPE 3 — KERNEL REPLACEMENT (algorithmic reformulation):
```

```
  This is not traditional fusion — it replaces an algorithm with a more
  HBM-efficient equivalent.

  Classic example: FlashAttention
    Before (standard attention):  O(S²) HBM traffic (materializes S×S matrix)
    After (FlashAttention):       O(S) HBM traffic  (tiles in SMEM, never
materializes)
    Mathematically identical; radically different HBM access pattern.

  Other examples:
    Online softmax: replaces two-pass softmax with streaming one-pass
    Fused layer normalization: replaces mean/variance/normalize/scale with one
pass
    INT4 dequant + GEMM: replaces separate dequant + FP16 GEMM with fused INT4
GEMM
```

9.2.2 Fusion Opportunity Analysis — LLaMA Transformer Block

```
UNFUSED vs FUSED: ONE LLAMA-3 TRANSFORMER BLOCK (LLaMA-3 70B, BF16):

  Operation chain (Pre-Norm transformer):

  UNFUSED (12 HBM round-trips for the non-GEMM path):
    1. x → RMSNorm → x_normed  [READ x, WRITE x_normed: 2 × 67 MB]
    2. x_normed → QKV_linear    [READ x_normed: 67 MB, GEMM output: 201 MB]
    3. Q, K, V = split(QKV)     [READ 201 MB, WRITE 3 × 67 MB = 201 MB]
    4. Q, K → RoPE              [READ/WRITE 2 × 67 MB = 134 MB]
    5. QKV → FlashAttn → attn   [READ 201 MB, WRITE 67 MB]
    6. attn → O_proj → attn_out [READ 67 MB (GEMM), WRITE 67 MB]
    7. x + attn_out → residual  [READ x+attn_out, WRITE residual: 3 × 67 MB]
    8. residual → RMSNorm2 → y  [READ/WRITE: 2 × 67 MB]
    9. y → FFN_gate + FFN_up    [READ y twice, WRITE gate+up: 3 × 67 MB]
    10. SiLU(gate) × up → act   [READ/WRITE gate+up+act: 3 × 67 MB]
    11. act → FFN_down → ffn    [GEMM READ/WRITE: 67 MB + 67 MB]
    12. residual + ffn → out    [READ/WRITE: 3 × 67 MB]

    Estimate non-GEMM HBM traffic: ~1.2 GB per block (excludes GEMM traffic)

  FUSED (production TRT-LLM / torch.compile):
    Fusion 1: QKV horizontal → x read once (saves 134 MB × 80 = 10.7 GB/pass)
    Fusion 2: RMSNorm + QKV linear → eliminates x_normed HBM write
    Fusion 3: RoPE fused into FlashAttention → no separate RoPE kernel
```

```
    Fusion 4: Residual add + RMSNorm2 → one kernel (saves 1 HBM write)
    Fusion 5: SiLU(gate) × up fused → gate and up results never touch HBM

    Estimated savings: 40-60% HBM traffic reduction for non-GEMM ops
    End-to-end inference speedup: 1.3-1.9× vs unfused PyTorch eager

HBM TRAFFIC QUANTIFIED (LLaMA-3 70B, S=4096, BF16, ONE LAYER):
  GEMM traffic (unavoidable — must load weights):
    QKV + O + FFN (W1+W3+W2): ~1.4 GB per layer
  Non-GEMM unfused traffic (avoidable with fusion):
    ~1.2 GB per layer  (86% of GEMM traffic — substantial!)
  Non-GEMM fused traffic:
    ~0.5 GB per layer  (fusion eliminates ~58% of avoidable traffic)
  Total savings per layer: ~0.7 GB
  80 layers: 80 × 0.7 = 56 GB per forward pass — eliminated by fusion alone
```

9.3 The PyTorch 2.x Compiler Stack

PyTorch 2.0 introduced `torch.compile` — a JIT compilation system that automatically applies fusion and other optimizations to arbitrary PyTorch models. It is a four-layer system, each layer with a distinct responsibility.

9.3.1 The Four-Layer Architecture

```
PYTORCH 2.x COMPILATION PIPELINE:

User code (Python PyTorch model)
        |
        ▼
┌──────────────────────────────────────────────────────────────────┐
| LAYER 1: TORCHDYNAMO (Python → FX Graph)                         |
|                                                                  |
|  Mechanism: Python BYTECODE INTERPRETER HOOKING                  |
|    Intercepts execution at the bytecode level.                   |
|    Records tensor operations as a static computation graph.      |
|    Falls back to Python for unsupported constructs.              |
|                                                                  |
|  GUARD SYSTEM:                                                   |
|    Records conditions for graph validity:                        |
|    "input shape is (32, 4096)", "model is in eval mode"          |
|    If guards fail (new shape): recompile for new shape.          |
|    Graph cache keyed by guard conditions.                        |
```

```
│                                                                  │
│  GRAPH BREAKS: when Dynamo can't trace through Python code       │
│    Causes: Python data structures, non-tensor control flow,      │
│            print statements, unsupported Python ops              │
│    Effect: compilation splits into sub-graphs; overhead grows    │
│    Diagnosis: TORCH_COMPILE_DEBUG=1; look for "graph break" log  │
└──────────────────────────────────────────────────────────────────┘
        │ FX graph (static computation graph)
        ▼
┌──────────────────────────────────────────────────────────────────┐
│ LAYER 2: AOTAutograd (Forward + Backward Graph)                  │
│                                                                  │
│  Traces BOTH forward AND backward pass in one step.              │
│  Normal PyTorch autograd: builds backward graph at RUNTIME       │
│    → Can't be optimized ahead of time.                           │
│  AOTAutograd: joint graph (forward + vjp for backward)           │
│    → Both passes can be optimized simultaneously.                │
│                                                                  │
│  ENABLES:                                                        │
│    Min-cut optimization (what to save vs recompute)              │
│    Fusion ACROSS forward-backward boundary                       │
│    Constant folding THROUGH the backward pass                    │
└──────────────────────────────────────────────────────────────────┘
        │ Optimized joint FX graph
        ▼
┌──────────────────────────────────────────────────────────────────┐
│ LAYER 3: TORCHINDUCTOR (Graph → Triton/C++)                      │
│                                                                  │
│  KEY OPTIMIZATIONS:                                              │
│    Operator fusion: groups fusable ops into single kernels       │
│    Tiling: selects tile sizes for SMEM efficiency                │
│    Vectorization: SIMD for CPU; vectorized loads for GPU         │
│    Loop ordering: optimizes memory access coalescing             │
│    Layout propagation: converts to Tensor Core-friendly layout   │
│                                                                  │
│  OUTPUT TARGETS:                                                 │
│    GPU:  Generates Triton kernels (Python-level CUDA)            │
│    CPU:  C++ with AVX-512 vectorization                          │
│    AMD:  Triton via ROCm backend                                 │
│                                                                  │
│  CACHING: Generated kernels cached to ~/.cache/torch/inductor/   │
│           Reused across runs if graph hash matches               │
└──────────────────────────────────────────────────────────────────┘
        │ Triton kernel code
```

```
        ▼
┌──────────────────────────────────────────────────────────────────┐
| LAYER 4: TRITON JIT (Triton → PTX/HSACO)                          |
|                                                                   |
|  Handles: SMEM allocation, warp scheduling, autotuning            |
|  Output: PTX (NVIDIA) or HSACO (AMD) assembly                    |
|  Auto-tunes: benchmarks BLOCK_M×BLOCK_N×BLOCK_K combinations    |
|  Caches: optimal configuration per shape                          |
└──────────────────────────────────────────────────────────────────┘
        |
        ▼
   GPU Execution (Tensor Cores / Matrix Cores)
```

9.3.2 torch.compile in Practice

```python
```

```python
import torch

model = LLaMAModel(config).cuda().bfloat16()

# THREE COMPILATION MODES — different aggressiveness:

# Mode 1: Default — balanced, fastest compile time
model_default = torch.compile(model)
# Best for: training, rapid iteration, first deployment

# Mode 2: reduce-overhead — CUDA Graphs + aggressive fusion
model_fast = torch.compile(model, mode="reduce-overhead")
# Best for: inference with fixed input shapes
# Overhead: 30-120s compile; saves 10-30% TPOT afterward
# Requirement: inputs must be same shape on each call (or shape set)

# Mode 3: max-autotune — exhaustive tile search
model_best = torch.compile(model, mode="max-autotune")
# Best for: production inference after shapes are stable
# Overhead: 300-1800s compile (benchmarks all tile configurations)
# Benefit: finds optimal cuBLAS algorithm and tile sizes for each op

# DYNAMIC SHAPES (variable-length inputs):
model_dynamic = torch.compile(model, dynamic=True)
# Slower than static but handles any input shape
# Recompiles when a NEW shape is seen; reuses graph for seen shapes
# Best for: training (variable batch size) or serving with diverse lengths

# INSPECT WHAT torch.compile IS DOING:
import torch._dynamo as dynamo
dynamo.explain(model)(input_ids)
# Shows: number of graph breaks, sub-graphs captured, reason for each break

# VIEW GENERATED TRITON CODE:
from torch._inductor import config
config.trace.enabled = True
config.trace.output_code = True  # saves generated Triton to /tmp/
model_compiled = torch.compile(model)
output = model_compiled(input_ids)  # /tmp/output_code.py shows Triton kernels

# WARMING UP (critical for benchmarking):
for _ in range(3):
```

```
    output = model_compiled(input_ids)  # first few calls include compile time
# Now benchmark — compilation overhead is amortized
```

9.3.3 torch.compile Performance Benchmarks

```
REAL-WORLD SPEEDUPS (from PyTorch published benchmarks):

  Model                Mode               Speedup   Primary Source
  ──────────────────────────────────────────────────────────────────
  BERT-large           default            1.28×     Operator fusion
  BERT-large           max-autotune       1.42×     + Tile size tuning
  LLaMA-2 7B (inf)    reduce-overhead    1.31×     CUDA Graphs dominant
  LLaMA-2 7B (train) default            1.24×     Min-cut recompute
  ResNet-50            default            1.18×     Less fusion opportunity
  ViT-L/16             default            1.35×     Attention fusion key
  ──────────────────────────────────────────────────────────────────


  Speedup source breakdown (approximate):
    Operator fusion:         ~40% of total gain
    CUDA Graphs (overhead):  ~30% of total gain
    Layout optimization:     ~15% of total gain
    Tile size tuning (auto): ~15% of total gain

WHEN torch.compile DOES NOT HELP:
  Workloads dominated by large GEMM (cuBLAS already optimal):
    torch.compile speedup: 5-10% (just overhead elimination, no fusion gain)
    Better approach: FP8 via Transformer Engine (2× throughput, zero code change)

  Frequent dynamic shapes (every call different shape):
    Guard misses trigger recompilation; overhead dominates gain.
    Better approach: bucket input lengths to fixed shapes (pad to power of 2)

  Many graph breaks:
    Each break returns to eager Python; compilation overhead not amortized.
    Diagnosis: torch._dynamo.explain() to count breaks and find causes.
    Fix: Rewrite the Python causing breaks, or accept eager execution there.

  Very short forward passes (< 50 tokens, batch=1):
    CUDA Graph overhead (first-call capture) not amortized.
    torch.compile adds 0.5-2ms compile overhead per unique shape.
```

9.4 XLA — The Functional Compiler (JAX / TPU)

XLA (Accelerated Linear Algebra) is Google's compiler for JAX and TensorFlow. Its design philosophy is fundamentally different from TorchInductor: where Inductor works on traced Python graphs, XLA works on HLO (High-Level Operations) — a purely functional, side-effect-free IR where all shapes are statically known at compile time.

9.4.1 XLA's HLO and Optimization Passes

```
XLA COMPILATION PIPELINE:

JAX/Python code:
  @jax.jit
  def transformer_layer(x, weights):
      x = jax.nn.rms_norm(x, weights['gamma'])
      qkv = jnp.dot(x, weights['qkv'])
      ...

  ↓ jax.jit traces execution (requires static shapes)

HLO (High-Level Operations):
  %x_normed = rms-norm(%x, %gamma)
  %qkv = dot(%x_normed, %qkv_weight)
  ...
  Properties: purely functional, no side effects, all shapes known

  ↓ XLA optimization passes (applied in order):

  Pass 1: Algebraic simplification  (x + 0 → x, x × 1 → x)
  Pass 2: Dead code elimination      (remove ops with unused output)
  Pass 3: Common subexpression elim  (compute shared subexpressions once)
  Pass 4: Instruction fusion         (elementwise ops merged into loops)
  Pass 5: Layout assignment          (choose row/column-major for TC)
  Pass 6: Rematerialization          (recompute vs store trade-off)
  Pass 7: Memory space assignment    (SMEM vs HBM assignment)

  ↓ Backend code generation

PTX / XLA HLO → CUDA → GPU execution
```

9.4.2 XLA vs TorchInductor — When to Use Which

```
COMPARISON: XLA vs TorchInductor
```

```
                    XLA (JAX)                TorchInductor (PyTorch)
──────────────────────────────────────────────────────────────────────
IR level             HLO (typed, shaped)     FX graph (Python-based)
Shape requirement    STATIC (all fixed)       DYNAMIC (guards + recompile)
Control flow         Functional only          Python fallback (graph break)
Compilation model    AOT (ahead-of-time)      JIT (just-in-time, cached)
First run overhead   Very high (~minutes)     Moderate (~seconds to minutes)
Production maturity  Very high (Google scale) High (PyTorch 2.2+)
Custom kernel support Limited (XLA custom ops) Natural (Triton via Inductor)
Debugging            Hard (HLO is opaque)     Better (FX graph inspectable)
Multi-device         TPU-native               GPU-native (CPU/Metal too)
Typical speedup      1.3-2× vs eager PyTorch  1.2-1.8× vs eager PyTorch
AMD GPU support      No                       Yes (via Triton ROCm)
──────────────────────────────────────────────────────────────────────


USE XLA WHEN:
  JAX-based models (Gemma, Gemini training at Google)
  TPU deployment: XLA is the ONLY compiler supported on TPU
  Fully static shapes with known batch sizes (embedding models, offline batch)
  Largest models at Google: Gemini/Gemma training runs on XLA/TPU exclusively

USE TORCHINDUCTOR WHEN:
  PyTorch models (majority of open-source LLMs: LLaMA, Mistral, Qwen, etc.)
  Dynamic shapes (variable sequence lengths, variable batch sizes)
  AMD GPU deployment (Triton ROCm backend)
  Rapid iteration with custom ops and experimental architectures
  Models with non-trivial Python control flow in the forward pass
```

9.5 TensorRT and TRT-LLM — Offline Production Compilation

TensorRT performs exhaustive offline compilation that produces a hardware-specific engine binary. Unlike torch.compile (JIT), TensorRT amortizes its long compilation time across millions of inferences in production.

9.5.1 TensorRT Optimization Passes

```
TENSORRT GRAPH OPTIMIZATION PIPELINE:

Input: ONNX model or PyTorch model via torch-tensorrt

PASS 1 — GRAPH CLEANING:
```

```
  Dead code elimination:      Remove ops whose outputs are never consumed.
  Constant folding:            Pre-compute all-constant expressions at build time.
    Example: embedding_scale = weight_A × weight_B (both constants)
    TRT evaluates this once, stores result. Zero runtime cost.
  Redundant tensor elimination: Remove unnecessary transpose/reshape ops.
    Example: transpose(transpose(x)) → identity (zero ops at runtime).

PASS 2 — LAYER FUSION (the most impactful pass):
  Vertical fusion (chain):    Merge sequential ops → one kernel.
    Conv → BatchNorm → ReLU → fused CBR kernel (classic from CV, applies to FFN)
    GEMM → add_bias → SiLU   → fused GEMM+activation
    RMSNorm → GEMM           → fused norm+projection

  Horizontal fusion (parallel): Merge parallel ops → wider kernel.
    Q_proj + K_proj + V_proj  → QKV_proj (3× wider, x read once)
    Multiple small GEMMs      → batched GEMM

  Attention fusion:            Entire attention block → single kernel.
    QKV projection + attention computation + output projection → one optimized
kernel.
    Includes FlashAttention as the attention implementation.
    This is the most impactful single fusion in LLM inference.

PASS 3 — PRECISION ASSIGNMENT:
  Per-layer precision: Assign FP8, INT8, or FP16 based on:
    User constraints (sensitive layers keep FP16/BF16)
    Calibration quality (if calibration data shows instability → fallback)
    Quality-of-service threshold (some layers need higher precision)
  Mixed-precision graph: different precision per layer based on sensitivity.

PASS 4 — KERNEL SELECTION (most time-consuming pass):
  For each op and each possible input shape: benchmark ALL available kernels.
    cuBLAS variants: 50+ GEMM algorithms per shape/precision combination.
    cuDNN variants: for attention and normalization.
    TRT built-in plugins: highly optimized custom kernels.
    User plugins: custom kernels registered by the user.
  Select the FASTEST verified kernel per shape.
  This pass takes 5-30 minutes for a 70B model.
  Result at inference: O(1) dispatch — no algorithm selection overhead ever.

PASS 5 — MEMORY PLANNING:
  Analyze tensor lifetimes: which tensors can share HBM allocations?
  In-place ops: eliminate intermediate allocations where safe.
```

```
  Peak memory estimation: guarantee model fits in available HBM.
  Result: 15-25% HBM reduction vs naive allocation.
```

9.5.2 TRT-LLM — Production Build Workflow

```python
```

```
# TRT-LLM BUILD WORKFLOW (LLaMA-3 70B, FP8, TP=2):

# Step 1: Convert HuggingFace checkpoint to TRT-LLM format
python convert_checkpoint.py \
    --model_dir ./llama3-70b-hf \
    --output_dir ./llama3-70b-trt-ckpt \
    --dtype bfloat16 \
    --tp_size 2

# Step 2: Build optimized TRT engine
trtllm-build \
    --checkpoint_dir ./llama3-70b-trt-ckpt \
    --output_dir ./llama3-70b-engine \
    --gpt_attention_plugin float16 \   # Fused attention plugin (FAv3)
    --gemm_plugin float16 \            # cuBLASLt selection
    --max_batch_size 64 \
    --max_input_len 4096 \
    --max_output_len 2048 \
    --quantization fp8 \               # FP8 W8A8 via TE
    --tp_size 2 \
    --workers 2                        # Parallel build (one per TP rank)

# Build time: 10-45 min for 70B. Output: engine_rank0.engine, engine_rank1.engine
# Artifacts are GPU-specific — rebuild when switching GPU model.

# Step 3: Serve with TRT-LLM runtime
python3 -m tensorrt_llm.serve \
    --engine_dir ./llama3-70b-engine \
    --tokenizer_dir ./llama3-70b-hf \
    --max_tokens_in_paged_kv_cache 32768

# Runtime optimizations active:
#   CUDA Graphs (decode path — captured at startup for each batch size)
#   Continuous batching (in-flight batching scheduler)
#   Paged KV cache (PagedAttention equivalent)
#   TP AllReduce overlap with next layer's GEMM (async streams)
#   FP8 GEMM via Transformer Engine

# SPEEDUP vs PyTorch eager (LLaMA-3 70B, H100, batch=32, FP8):
#   PyTorch eager BF16:   ~850 tok/s/GPU
#   torch.compile BF16:  ~1,100 tok/s/GPU  (+29%)
```

```
#   TRT-LLM FP8:          ~2,200 tok/s/GPU  (+159%)
#   Combined TRT-LLM gain: 2.6× vs PyTorch eager
```

9.6 NVFuser — NVIDIA's Runtime Fusion Engine

NVFuser is NVIDIA's production fusion compiler embedded directly in PyTorch and TRT-LLM. Unlike TorchInductor (which generates Triton), NVFuser generates CUDA C++ code and exploits NVIDIA-specific hardware features directly.

9.6.1 NVFuser vs TorchInductor

```
NVFUSER vs TORCHINDUCTOR:

                    NVFuser                    TorchInductor + Triton
  ──────────────────────────────────────────────────────────────────────
  Targets           NVIDIA GPU only            NVIDIA + AMD + CPU
  Code generation   CUDA C++ (direct)          Triton (Python-level)
  Integration       PyTorch core (automatic)   torch.compile frontend
  Fusion scope      Pointwise + reduction       Pointwise + reduction + GEMM
  SMEM management   Manual (expert control)    Automatic (Triton handles)
  Performance ceil  Higher (direct CUDA)       Slightly lower
  Debuggability     Hard (generated C++)        Better (Triton readable)
  Custom ops        Limited                     Full Triton extensibility
  ──────────────────────────────────────────────────────────────────────

  NVFuser wins for:
    Complex pointwise + reduction chains (RMSNorm, BatchNorm, Softmax)
    Patterns requiring careful SMEM tiling for maximum bandwidth
    Cases where Triton's SMEM management leaves performance on the table

  TorchInductor wins for:
    AMD GPU deployment (Triton is cross-platform)
    Custom operators (extend with Triton easily)
    Training (tighter AOTAutograd integration)
```

9.6.2 Critical NVFuser Fusions in LLM Production

```
KEY NVFUSER FUSIONS IN TRT-LLM (LLaMA-3 70B):

1. RMSNorm Fusion:
   Operations: sum(x²), sqrt(sum/N + ε), divide, multiply_by_weight
```

```
   Unfused: 5 kernels, 10 HBM round-trips
   NVFused: 1 kernel, 2 HBM round-trips (x in → x_normed out)
   Speedup: ~4× vs unfused
   Per-layer savings (2 RMSNorms, BF16, S=4096, d=8192): 4 × 67 MB = 268 MB
   80 layers: 80 × 268 MB = 21.4 GB saved per step (the §2.9.4 calculation)

2. SwiGLU Activation (W1/W3 gate fusion):
   Operations: gate = SiLU(W3 × x); output = gate × (W1 × x)
   NVFused: Read gate + up → compute SiLU in registers → multiply → write
   Eliminates: intermediate HBM write for SiLU output
   Speedup: ~2× vs separate ops

3. Residual + RMSNorm:
   Operations: x_out = x + residual; x_norm = RMSNorm(x_out)
   NVFused: reads x + residual, adds, normalizes, writes — one kernel
   Eliminates: HBM write for residual sum before normalization

4. Attention online softmax:
   Streaming softmax via running max/sum statistics.
   Eliminates full S×S score matrix materialization.
   Core mechanism of FlashAttention (available as NVFuser primitive too).
```

9.7 Graph-Level Optimizations in Depth

9.7.1 Dead Code Elimination, Constant Folding, CSE

```
DEAD CODE ELIMINATION (DCE):

  Training code run at inference:
    x1 = linear(x)           # used
    x2 = dropout(x1, p=0.1)  # UNUSED at inference (dropout is eval-disabled)
    x3 = layer_norm(x1)       # used

  After DCE:
    x1 = linear(x)
    x3 = layer_norm(x1)       # x2 eliminated entirely

  Practical importance: training code often contains monitoring ops,
  gradient hooks, and conditional dropout that are dead at inference.
  DCE removes them without requiring the user to maintain separate codebases.
```

```
CONSTANT FOLDING:

  Before:
    scale = head_dim ** -0.5  # = 128 ** -0.5 = 0.08838...
    scores = (q @ k.T) * scale  # scale computed every step!

  After constant folding:
    scale = 0.08838...  # evaluated once at compile time
    scores = (q @ k.T) * 0.08838...  # runtime: just multiply, no pow op

  TensorRT constant folding: evaluates ANY op whose inputs are all constants.
  Includes: weight transformations, positional encoding tables (if fixed),
            all normalization statistics for BatchNorm (in inference mode).

COMMON SUBEXPRESSION ELIMINATION (CSE):

  Before:
    a = compute_expensive(x)  # expensive_op called once
    b = a + 1.0
    c = a + 2.0               # compute_expensive(x) called AGAIN!
    d = a * 0.5               # compute_expensive(x) called AGAIN!

  After CSE:
    a = compute_expensive(x)  # called exactly once; b, c, d reuse result
    b = a + 1.0
    c = a + 2.0
    d = a * 0.5

  LLM relevance: Attention score Q × K^T may be needed for both attention
  output computation and for gradient computation. CSE ensures the matrix
  product is computed once.
```

9.7.2 Layout Propagation — The Invisible Performance Multiplier

```
LAYOUT PROPAGATION:

  Problem: PyTorch default tensor layout (row-major / NCHW) may be suboptimal
  for Tensor Core performance or specific fusion patterns.

  Tensor Core requirement (H100 BF16 WGMMA):
    Matrix A: must be in row-major (M × K) layout
    Matrix B: must be in column-major (K × N) layout for maximum TC throughput
              (or use specific "interleaved" format for 16-byte-aligned access)
```

```
  Layout mismatch consequence:
    GEMM with wrong layout: transpose required before GEMM → extra HBM round-trip
    Cost: 1 extra HBM read + 1 extra HBM write (134 MB × 80 layers = 10.7 GB/pass)

  WHAT LAYOUT PROPAGATION DOES:
    Analyzes the entire computation graph.
    Assigns layouts to each tensor to minimize total transpose operations.
    Inserts transposes only at points where the cost is minimal.
    For GEMM chains (the entire LLM): optimal layout often eliminates all
transposes.

  PyTorch channels-last (NHWC) — the vision equivalent:
    import torch
    model = model.to(memory_format=torch.channels_last)  # vision models
    # Convolution kernels get ~20-40% speedup from matching TC-native layout
    # TorchInductor propagates this layout automatically

  For LLM GEMM:
    TorchInductor and TRT-LLM both apply layout propagation automatically.
    Users see the benefit without any code changes.
```

9.8 Min-Cut Recomputation — Fusion and Activation Checkpointing

This is the intersection of compiler optimization and distributed training memory management. Understanding it is essential for scaling LLM training beyond 7B parameters.

9.8.1 The Activation Memory Problem

```
TRAINING MEMORY BREAKDOWN (LLaMA-3 70B, batch=1, S=4096):

  Activation memory (store everything for backward):
    Per transformer block: ~S × d_model × ~16 tensors × 2B
                           = 4096 × 8192 × 16 × 2 = 1.07 GB per block
    80 blocks: 80 × 1.07 GB = 85.6 GB activations!

  Plus model + optimizer state: ~1,100 GB (from Chapter 2)
  Total: 1,185 GB → needs 15 H100s just for one training sequence!

NAIVE ACTIVATION CHECKPOINTING:
  Discard ALL intermediate activations after forward pass.
  Recompute everything needed for backward from the checkpoint.
```

```
  Memory: O(sqrt(N)) vs O(N) — cubic reduction.
  Cost: ~30% extra compute (recomputing the forward pass during backward).

  This is the brute-force approach. The compiler can do much better.
```

9.8.2 Min-Cut Recomputation — The Compiler Approach

```
MIN-CUT RECOMPUTATION (AOTAutograd in PyTorch 2.x):

  INSIGHT: Not all activations are equally expensive to recompute.
    Fused RMSNorm: forward cost = 1 HBM read → nearly FREE to recompute.
    FlashAttention: forward cost = 1 pass over Q,K,V → expensive to recompute.
    Large GEMM output: forward cost = expensive → ALWAYS store, never recompute.

  FUSION CHANGES THE RECOMPUTATION COST:
    Unfused RMSNorm: forward costs 5 HBM reads → expensive to recompute.
    Fused RMSNorm:   forward costs 1 HBM read  → nearly FREE to recompute.

    Fusing RMSNorm makes it a prime candidate for recomputation.
    The memory saved (67 MB per norm × 160 norms = 10.7 GB) is large.
    The recompute cost (one HBM read each) is negligible.

  THE MIN-CUT ALGORITHM:
    Model the computation graph as a min-cut problem:
      Nodes: operations
      Edges: data flow with weights = memory size
      Cut: partition into {save set, recompute set}

    Find the partition that minimizes:
      Total memory (of save set) + Total recompute cost
    Subject to: memory budget constraint (target GPU HBM)

    Solved via max-flow / min-cut algorithms.
    AOTAutograd runs this at compile time; result is fixed for all training steps.

  PRACTICAL RESULTS:
    Naive recompute all:     60% memory reduction, 30% extra compute
    Min-cut (PyTorch 2.x):  60% memory reduction, < 5% extra compute
    Savings vs naive: 25% compute overhead eliminated while same memory.

  ENABLING IN PYTORCH:
    from torch.utils.checkpoint import checkpoint_wrapper, CheckpointImpl
    from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
```

```
    # Apply selective checkpointing to transformer layers
    model = FSDP(
        checkpoint_wrapper(
            model,
            checkpoint_impl=CheckpointImpl.NO_REENTRANT  # AOTAutograd-compatible
        )
    )

    # torch.compile automatically applies min-cut to decide WHAT to recompute:
    model = torch.compile(model, mode="default")
    # AOTAutograd handles the save/recompute split; user doesn't manage it.

  WHICH TENSORS DOES MIN-CUT TYPICALLY SELECT FOR RECOMPUTATION?
    RECOMPUTE (cheap to recompute):
      RMSNorm outputs (when fused)
      Attention mask (pure arithmetic, no HBM load)
      Dropout masks (deterministic from seed)
      Softmax outputs (when FlashAttention recomputes during backward)

    SAVE (expensive to recompute):
      GEMM inputs (loaded full model weights for each recompute — too expensive)
      KV cache (used across many backward steps)
      First-layer activations (input embeddings — loaded once, reused)
```

9.9 CUDA Graphs — Eliminating CPU Dispatch Overhead

CUDA Graphs were introduced in Chapter 7 from the kernel perspective. This section examines them from the compiler architecture perspective — how they interact with torch.compile, TRT-LLM, and dynamic batching.

9.9.1 The Dispatch Overhead Problem — Quantified

```
CPU KERNEL LAUNCH OVERHEAD — LLaMA-3 70B DECODE:

  One transformer layer: ~25-40 kernel launches
    QKV GEMM:              1 kernel
    FlashAttention:        1 kernel
    Output GEMM:           1 kernel
    RMSNorm (fused):       1 kernel
    Residual add:          1 kernel (or fused with RMSNorm)
    FFN GEMMs × 3:         3 kernels
```

```
    SiLU (fused with W3):  0 extra
    AllReduce (TP=8):      1 NCCL call
    ... (misc elementwise): 5-10 more
  Total: ~15-20 kernels per layer after fusion

  80 layers: 80 × 17 = 1,360 CUDA API calls per decode step.
  Python + CUDA driver overhead per call: ~5-10 μs
  Total dispatch overhead: 1,360 × 7 μs = 9.5 ms per decode step!

  AT TARGET TPOT = 30 ms:
    GPU compute: ~20-22 ms
    CPU dispatch: 9.5 ms
    Total: ~30 ms
    CPU overhead fraction: 9.5/30 = 32% — nearly a THIRD of TPOT is pure overhead!

  WITH CUDA GRAPHS:
    CPU overhead: 1 cudaGraphLaunch() call × ~10 μs = 0.01 ms
    Total: ~20-22 ms (compute only)
    TPOT improvement: 32% reduction
    Note: This is purely overhead elimination — zero change in GPU work done.
```

9.9.2 CUDA Graph Interaction with Dynamic Batching

```
THE STATIC SHAPE REQUIREMENT — THE CORE CHALLENGE:

  CUDA Graphs capture kernel arguments INCLUDING tensor addresses and shapes.
  If shape changes: addresses may differ → graph replay produces wrong results
  or crashes.

  VLLM APPROACH — MULTIPLE GRAPHS PER BATCH SIZE:
    Capture one graph per batch size: [1, 2, 4, 8, 16, 32, 64, 128, 256]
    Startup overhead: 9 captures × ~60s each = ~9 minutes startup time.
    At inference: select graph matching current batch size.
    For batch sizes not in the list: fall back to eager (no graph).

    Padding strategy: batch=5 → use batch=8 graph, pad with dummy inputs.
    Tradeoff: slightly more compute (dummy tokens do real work) vs graph benefit.

  TRT-LLM APPROACH — OPTIMIZATION PROFILES:
    TRT-LLM builds multiple optimization profiles at engine build time:
    Profile for [min_batch, optimal_batch, max_batch] × [min_seq, opt_seq,
max_seq].
    Engine contains CUDA Graphs for each profile.
```

```
    No runtime capture needed; profiles baked into .engine binary.
    Tradeoff: longer build time, larger engine file.

  WHEN CUDA GRAPHS CANNOT BE USED:
    Dynamic control flow that depends on tensor values
    Variable-size allocationsduring the forward pass
    Custom Python logic interleaved with GPU kernels
    Speculative decoding (draft acceptance is dynamic → graph incompatible)
    → For spec-decode: CUDA Graphs only for the verification step (fixed shape).
```

9.10 Compiler Selection Framework

9.10.1 Decision Guide — Which Compiler for Which Use Case

```
COMPILER SELECTION DECISION TABLE:

  Use Case                        Primary       Secondary      Avoid
  ──────────────────────────────────────────────────────────────────────
  PyTorch inference, static       TRT-LLM       torch.compile  Eager
  shapes, NVIDIA GPU              (offline)     (reduce-ovhd)
  ──────────────────────────────────────────────────────────────────────
  PyTorch inference, dynamic      torch.compile TRT-LLM w/      Eager for
  shapes / rapid iteration        (dynamic=True) dynamic prof   hot path
  ──────────────────────────────────────────────────────────────────────
  JAX model, TPU deployment       XLA/jax.jit   —              torch.compile
                                                                (no TPU support)
  ──────────────────────────────────────────────────────────────────────
  AMD GPU (MI300X), PyTorch       torch.compile  —             TRT-LLM
                                  (Triton ROCm)                (NVIDIA-only)
  ──────────────────────────────────────────────────────────────────────
  LLM training, memory-critical   torch.compile  XLA for JAX   TRT-LLM
                                  + min-cut                     (inference only)
  ──────────────────────────────────────────────────────────────────────
  Custom op development           Triton kernel  torch.compile  TRT-LLM
  (new attention variant)         + torch.compile               (limited plugin)
  ──────────────────────────────────────────────────────────────────────
  Stable production serving       TRT-LLM        —             Eager
  (model rarely changes)          (+ CUDA Graphs)
  ──────────────────────────────────────────────────────────────────────

  PERFORMANCE BENCHMARK COMPARISON (LLaMA-3 70B, H100, batch=32, FP8):
```

```
    PyTorch eager BF16:          ~850 tok/s/GPU
    torch.compile default BF16:  ~1,100 tok/s/GPU  (+29%)
    TRT-LLM FP8:                  ~2,200 tok/s/GPU  (+159%)
    TRT-LLM FP8 + CUDA Graphs:   ~2,600 tok/s/GPU  (+206%)

    Note: TRT-LLM advantage is not just compilation — FP8 contributes ~2×
    and the CUDA Graph and fusion stack contributes the remainder.
    Controlled comparison (same precision, both compiled):
      torch.compile BF16 vs TRT-LLM BF16: TRT-LLM ~1.3-1.5× faster
      Difference: exhaustive kernel selection + offline fusion pipeline.
```

9.10.2 Measuring Compiler Impact Correctly

```
CORRECT BENCHMARKING METHODOLOGY FOR COMPILERS:

  COMMON MISTAKES:
    1. Timing the first call (includes compilation overhead → misleading).
    2. Not warming up the GPU (clock frequency not at boost).
    3. Comparing torch.compile to TRT-LLM at different precisions.
    4. Measuring wall-clock without CUDA synchronization.

  CORRECT APPROACH:
    import torch, time

    # 1. Warmup (runs compilation and JIT tuning)
    for _ in range(5):
        out = model(inputs)
    torch.cuda.synchronize()

    # 2. Timed benchmark (wall-clock with sync)
    N = 50
    start = time.perf_counter()
    for _ in range(N):
        out = model(inputs)
    torch.cuda.synchronize()
    elapsed = (time.perf_counter() - start) / N

    # 3. Same precision in both conditions:
    # eager_bfloat16 vs compiled_bfloat16  (isolates compiler effect)
    # NOT: eager_bfloat16 vs compiled_fp8  (mixes compiler + quantization)

  WHAT TO MEASURE:
    Throughput (tokens/sec): primary for serving
```

```
    TPOT P50 and P99: for latency comparison
    MFU: for training efficiency

  READING RESULTS:
    If speedup < 1.1×:  compiler overhead dominates; model may not benefit.
    If speedup 1.1-1.5×: good fusion gain; worth deploying.
    If speedup > 1.5×:   significant overhead-bound before (CUDA Graphs helped a
lot).
    If slowdown:          graph breaks or dynamic shapes; investigate with
dynamo.explain().
```

9.11 Key Takeaways and Review Questions

```
CHAPTER 9 CORE PRINCIPLES:

  1. FUSION IS THE MASTER OPTIMIZATION:
     Every other compiler technique — constant folding, layout propagation,
     min-cut recomputation — either enables fusion or removes obstacles to it.
     A model running 12 separate kernels for what could be 3 is fundamentally
     memory-bandwidth-bound regardless of how good those 12 kernels are.

  2. THE THREE REGIMES DETERMINE THE CORRECT COMPILER TOOL:
     Overhead-bound: CUDA Graphs (torch.compile reduce-overhead or TRT-LLM).
     Memory-bound: Fusion (TorchInductor vertical fusion, NVFuser patterns).
     Compute-bound: Kernel selection (TRT-LLM Pass 4, tile tuning, FP8).
     Applying a compute-bound fix to an overhead-bound problem does nothing.

  3. TORCH.COMPILE IS NOT ALWAYS THE RIGHT ANSWER:
     20-50% speedup for small-kernel, fusion-friendly models.
     5-10% speedup for GEMM-dominated workloads (cuBLAS already optimal).
     Potential regression for dynamic shapes or frequent graph breaks.
     Profile first; compile second.

  4. TRT-LLM IS THE PERFORMANCE CEILING FOR NVIDIA PRODUCTION INFERENCE:
     2.6× over PyTorch eager = the sum of FP8 (2×) + fusion (1.3×) + overhead
elim.
     Requires: stable model, NVIDIA GPU, offline build time.
     Worth it for: high-volume production serving (amortized over millions of
requests).

  5. MIN-CUT BEATS NAIVE CHECKPOINTING BY 25% COMPUTE:
```

```
     Naive recompute-all: 30% extra compute for 60% memory reduction.
     Min-cut (PyTorch 2.x with torch.compile): < 5% extra compute for 60% memory
reduction.
     Key insight: fused ops become free to recompute. Always use torch.compile
with training.

  6. CUDA GRAPHS ARE MANDATORY FOR LOW-LATENCY SERVING:
     At TPOT = 30ms: dispatch overhead = 32% of TPOT without CUDA Graphs.
     With CUDA Graphs: < 1% overhead. Mandatory for any production serving system.
     Limitation: static shapes required. Handle with per-batch-size graph capture.

COMPILER SPEEDUP REFERENCE:
  PyTorch eager:             baseline
  torch.compile default:    +15-30%
  torch.compile reduce-ovhd: +20-40% (adds CUDA Graphs)
  torch.compile max-autotune: +30-50% (adds tile search)
  TRT-LLM (same precision): +30-50% over torch.compile
  TRT-LLM FP8 vs eager BF16: ~2.6× total
  XLA on TPU:                comparable to TorchInductor on GPU
```

Review Questions (from r3 integration)

Difficulty: Foundational

Question 1 (Fusion Opportunity Identification): A custom transformer layer applies: `output = LayerNorm(x + Dropout(Linear(GELU(Linear(x)))))`. Profile shows 12 separate CUDA kernels and 18% of H100 theoretical HBM bandwidth. (a) Identify every fusion opportunity — which ops can be fused and why. (b) Estimate the theoretical speedup from optimal fusion.

Expected answer: (a) Fusion opportunities in order: - Linear(x) → GELU → single kernel: GELU is elementwise, executes element-by-element immediately after GEMM output. Fuse: GEMM + GELU in one kernel, GELU result stays in registers. Eliminates 1 HBM round-trip. - GELU output → Linear → single kernel: GELU result is the input to the next GEMM. With previous fusion this is already fused. Alternatively: GELU + Linear = fused activation-GEMM (less common but possible with Triton). - Dropout → elementwise on Linear output: stochastic zero-masking. Fuse with Linear output: after GEMM, apply dropout mask in same kernel before writing to HBM. Eliminates 1 HBM round-trip. - x + dropout_output → residual add: elementwise addition. Fuse with dropout: compute dropout × output + x in one pass. Eliminates 1 more HBM round-trip. - Residual + LayerNorm: the residual result feeds directly into LayerNorm. Canonical fusion: compute residual, normalize, write once. Eliminates 1 HBM round-trip. Total unfused kernels: ~12. After fusion: Linear1 + GELU, Linear2 + Dropout + Residual, LayerNorm = ~4 kernels. Saves ~8 HBM round-trips. (b) At 18% of theoretical HBM bandwidth: the unfused system is doing ~5.5× more HBM traffic than the minimum (18% util means the compute is fast but memory traffic is the problem). After fusing 8/10 round-trips: HBM traffic drops to ~20% of original →

potential 5× improvement in the non-GEMM path. Since GEMMs are typically 70-80% of total time and are already near-optimal: total speedup ≈ 1 / (0.70 + 0.30/5) = 1 / 0.76 = 1.32× — approximately 1.3× total from fusion alone.

Difficulty: Intermediate

Question 2 (Compiler Selection): You have three scenarios. For each, specify which compiler to use and why: (a) LLaMA-3 70B PyTorch inference on 4× H100, serving 10,000 requests/day. Model is stable (no architecture changes planned for 6 months). (b) Research model with custom attention variant (sparse attention + sliding window). Running experiments on 1× H100; changing architecture every 2-3 days. (c) Gemma 7B training on 64× TPU v5p pods at Google.

Expected answer: (a) TRT-LLM. Rationale: stable model, H100 hardware, high volume (10K req/day amortizes 45-min build time easily). TRT-LLM delivers 2.6× vs eager through FP8 + offline optimization + CUDA Graphs. The 45-minute one-time build time is justified. torch.compile would be 30-50% slower than TRT-LLM for production NVIDIA inference at this scale. (b) torch.compile with mode="default" or custom Triton kernels. Rationale: custom attention → TRT-LLM requires custom plugins (complex to write, rebuild on every change). torch.compile handles arbitrary PyTorch code including custom ops written in Triton. Architecture changes every 2-3 days → avoid build-time compilers entirely. Trade-off: ~1.3-1.5× slower than TRT-LLM, but zero friction in the research loop. (c) XLA via jax.jit. Rationale: TPU v5p only supports XLA as the compiler — there is no CUDA, no torch.compile, no TRT-LLM on TPU. Gemma is a Google model trained in JAX. XLA is the only option and is highly optimized for TPU. Additionally, Google's TPU pods benefit from XLA's static-shape compilation: training batch sizes and sequence lengths are fixed, eliminating the dynamic-shape overhead that limits XLA on GPU.

Difficulty: Principal Level

Question 3 (torch.compile Architecture): An engineer runs torch.compile on a production LLaMA-3 70B serving pipeline and observes: first request takes 94 seconds (includes compile time), subsequent requests run at 1.26× the eager baseline. The engineer reports "torch.compile adds 94 seconds startup but only gives 26% speedup — not worth it." (a) Diagnose what the 94 seconds includes and how to address it. (b) Identify two reasons why 26% might be below expected speedup, and provide diagnostic steps. (c) Evaluate whether the engineer's conclusion is correct.

Expected answer: (a) The 94 seconds = TorchDynamo tracing + TorchInductor code generation + Triton JIT compilation + CUDA Graph capture (if reduce-overhead mode). This is a one-time cost per unique computation graph + shape combination. Fix: warm up the model during serving startup with representative inputs BEFORE serving the first user request. Standard practice: 5-10

warmup passes during initialization. Also: cache compiled kernels to disk (default behavior in PyTorch 2.x with ~/.cache/torch/inductor/) so subsequent deployments skip compilation. With caching: first request after initial compilation takes ~200ms, not 94 seconds. (b) Two likely reasons for below-expected speedup: (1) Graph breaks: the pipeline may include Python logic (attention mask generation, dynamic routing, sampling logic) that TorchDynamo cannot trace. Each break returns to eager Python and introduces overhead. Diagnosis: torch._dynamo.explain(model)(input) — look for count of graph breaks and reason for each. Fix: simplify or remove Python in the forward path; use torch.jit.trace for static sub-graphs. (2) Workload dominated by large GEMM (cuBLAS already optimal): torch.compile's fusion benefit is highest for small elementwise operations. If the LLaMA model is already using FlashAttention and fused GEMM+activation, torch.compile adds little beyond CUDA Graphs. Diagnosis: profile with nsys before and after torch.compile — if GEMM time is unchanged and the speedup comes entirely from reduced CPU overhead, the 26% is actually the correct ceiling for this workload. (c) The engineer's conclusion is incorrect in two ways: (1) The 94-second cost is amortized. At 10,000 requests/day (a modest serving load), 94 seconds = 0.94% of one day. If serving for 30 days: 94/(30×86,400) = 0.0036% overhead. A 26% speedup compounding over 30 days of serving completely dominates the one-time 94-second compilation cost. (2) The comparison should be cost-per-token, not just speedup. 1.26× speedup on the compute = 1/1.26 = 79% of original compute time = 21% cost reduction. At scale: 26% fewer GPU-hours × $1.66/GPU-hr = significant savings. The correct question is: does 26% throughput improvement on the serving fleet justify the 94-second startup overhead? For any serving system handling > ~100 requests/day: yes, decisively.

Difficulty: Principal Level

Question 4 (Min-Cut Recomputation vs Naive Checkpointing): A 70B LLM training job on 8×H100 (ZeRO-3) runs with naive activation checkpointing (recompute ALL activations) to fit in memory. MFU is 38%. Training loss curves look normal. A colleague proposes "just remove checkpointing — we have enough memory." Another proposes "use PyTorch 2.x min-cut recomputation." (a) Explain why naive recompute-all is conservative relative to min-cut, quantifying the compute overhead difference. (b) What changes when fused ops (RMSNorm, SiLU gate) are available in the graph? (c) What would you do, and what MFU improvement do you expect?

Expected answer: (a) Naive recompute-all discards every intermediate activation after forward, then recomputes all of them from checkpoints during backward. This adds ~30% extra compute (a second partial forward pass for each layer during backward). Min-cut (AOTAutograd) uses graph analysis to find which activations are cheapest to recompute. It creates an optimal partition: save expensive-to-recompute tensors (GEMM outputs, large tensors that require full model weight loads to recompute) and recompute cheap ones (elementwise ops, norms). Typical min-cut: ~5-8% extra compute vs 30% for naive. The 25% compute overhead gap directly

translates to MFU: if min-cut costs 6% extra vs naive 30%, that's ~24% more compute time recovered. At current 38% MFU with naive: min-cut could yield ~38% × (1 - 0.06) / (1 - 0.30) ≈ 51% MFU — a significant gain. (b) When fused ops are available: fused RMSNorm reads x once and writes x_normed. Recomputation cost = 1 HBM read (load x again) + registers for the computation. This is nearly free — the only cost is bandwidth for re-reading x, which is small relative to the memory saved by discarding the intermediate. Min-cut algorithm sees: "recompute cost of fused RMSNorm ≈ 0; memory saved = d_model × S × 2B = 67 MB per occurrence × 160 norms = 10.7 GB." This is an obviously favorable trade — min-cut always selects fused norms and elementwise ops for recomputation. Unfused norms would be more expensive to recompute and might be selected for saving instead. (c) Recommendation: enable torch.compile with default mode + activation checkpointing wrapper using CheckpointImpl.NO_REENTRANT (AOTAutograd-compatible). Do NOT simply remove checkpointing — without it, 85 GB of activations (from the §9.8.1 calculation) would be needed beyond the model/optimizer state, likely exceeding H100 memory with ZeRO-3. The "remove checkpointing" suggestion is likely to OOM. Expected MFU improvement: from 38% to approximately 48-52%, primarily from eliminating the 24% extra compute overhead of naive vs min-cut recomputation. Also: torch.compile's fusion improvements add another 5-10% MFU independently.

Further Reading

1. *Making Deep Learning Go Brrr From First Principles* — Horace He (2022). `horace.io/brrr_intro.html` The clearest treatment of the three performance regimes and how compilers address them. Required reading before any compiler investigation.
2. *TorchDynamo/TorchInductor Design Documentation* — PyTorch Team (2023). `pytorch.org/docs/stable/torch.compiler_deepdive.html` The authoritative guide to torch.compile internals. Read the "graph breaks" section and the Inductor lowering documentation.
3. *Optimizing Production PyTorch Models' Performance with Graph Transformations* — Meta AI Research (2022). `arxiv.org/abs/2210.06028` The AOTAutograd + min-cut recomputation paper. The memory savings derivation is exact.
4. *XLA: TensorFlow, Compiled* — Google Brain (2017). `arxiv.org/abs/1901.04645` The foundational XLA paper. HLO design and pass structure.
5. *NVIDIA TensorRT Optimization Guide* (2024). `docs.nvidia.com/deeplearning/tensorrt/developer-guide/` Read Chapter 8 (Layer Fusion) and Chapter 9 (Precision Modes). The kernel selection methodology (Pass 4) is documented in detail.

6. *CUDA Graphs* — NVIDIA Documentation (2023). `docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs` The definitive reference for CUDA Graph capture, replay, and limitations.

— End of Chapter 9 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved